WEEK 9 – HOSPITAL SALES AND PATIENT ANALYTICS SYSTEM

CITYCARE HOSPITAL MANAGEMENT SYSTEM

1. INTRODUCTION

The Hospital Sales and Patient Analytics System is an important module of the CityCare Hospital Management System. It is designed to analyse medicine sales, patient purchasing behaviour, order transactions, and revenue trends using SQL queries.

The system helps hospital management understand the total number of patient orders, medicine sales performance, patient spending patterns, and category-wise sales.

This module implements SQL aggregate functions such as COUNT(), SUM(), AVG(), MIN(), and MAX(), along with GROUP BY, HAVING, JOIN, ORDER BY, and subqueries to generate analytical reports.

2. OBJECTIVES

The main objectives of this module are:

- To implement SQL aggregate functions.
- To calculate total orders and sales revenue.
- To determine average, highest, and lowest order values.
- To analyse patient purchasing behaviour.
- To identify patients with maximum spending.
- To identify frequently purchasing patients.
- To analyse best-selling and least-selling medicines.
- To calculate category-wise medicine sales.
- To generate hospital sales and patient analytics reports.

3. TABLES USED

Table Name	Description
Patient	Stores patient details
Patient_Orders	Stores patient order information

Order_Details	Stores ordered medicine quantities and prices
Medicine	Stores medicine information and categories
Hospital_Payment	Stores payment transaction details

The main analysis uses Patient, Patient_Orders, Order_Details, and Medicine tables.

4. APPLY SQL AGGREGATE FUNCTIONS

4.1 Demonstrate Aggregate Functions

SQL Query:

```
SELECT
    COUNT(*) AS Total_Medicines,
    SUM(Unit_Price) AS Sum_Of_Prices,
    ROUND(AVG(Unit_Price), 2) AS Average_Price,
    MIN(Unit_Price) AS Lowest_Price,
    MAX(Unit_Price) AS Highest_Price
FROM Medicine;
```

Explanation:

This query demonstrates five SQL aggregate functions. COUNT() calculates the total number of medicines, SUM() adds medicine prices, AVG() calculates the average price, MIN() identifies the lowest price, and MAX() identifies the highest price.

Output:

```
SQL> SELECT
  2      COUNT(*) AS Total_Medicines,
  3      SUM(Unit_Price) AS Sum_Of_Prices,
  4      ROUND(AVG(Unit_Price), 2) AS Average_Price,
  5      MIN(Unit_Price) AS Lowest_Price,
  6      MAX(Unit_Price) AS Highest_Price
  7  FROM Medicine;

TOTAL_MEDICINES SUM_OF_PRICES AVERAGE_PRICE LOWEST_PRICE HIGHEST_PRICE
--------------- ------------- ------------- ------------ -------------
              6            35          5.83          2.5            12

SQL>
```

5. GENERATE TOTAL SALES REPORTS

5.1 Total Number of Orders Placed

SQL Query:

```
SELECT COUNT(*) AS Total_Orders
FROM Patient_Orders;
```

Explanation:

This query calculates the total number of orders placed by patients using the COUNT() function.

Output:

```
SQL> SELECT COUNT(*) AS Total_Orders
  2  FROM Patient_Orders;

TOTAL_ORDERS
------------
           4
```

5.2 Total Sales Revenue

SQL Query:

```
SELECT NVL(SUM(Total_Amount), 0) AS Total_Sales
FROM Patient_Orders;
```

Explanation:

This query calculates the total recorded order value using the SUM() function. NVL() replaces a NULL result with zero.

Output:

```
SQL> SELECT NVL(SUM(Total_Amount), 0) AS Total_Sales
  2  FROM Patient_Orders;

TOTAL_SALES
-----------
       45.5
```

5.3 Average Order Value

SQL Query:

```
SELECT ROUND(AVG(Total_Amount), 2)
    AS Average_Order_Value
FROM Patient_Orders;
```

Explanation:

This query calculates the average order value and rounds the result to two decimal places.

Output:

```
SQL> SELECT ROUND(AVG(Total_Amount), 2) AS Average_Order_Value
  2  FROM Patient_Orders;

AVERAGE_ORDER_VALUE
-------------------
              11.38
```

5.4 Highest and Lowest Order Amount

SQL Query:

```
SELECT
    MAX(Total_Amount) AS Highest_Order,
    MIN(Total_Amount) AS Lowest_Order
FROM Patient_Orders;
```

Explanation:

This query identifies the highest and lowest order amounts using MAX() and MIN().

Output:

```
SQL> SELECT
  2      MAX(Total_Amount) AS Highest_Order,
  3      MIN(Total_Amount) AS Lowest_Order
  4  FROM Patient_Orders;

HIGHEST_ORDER LOWEST_ORDER
------------- ------------
           13           10
```

5.5 Total Sales for a Specific Period

SQL Query:

```
SELECT NVL(SUM(Total_Amount), 0) AS Period_Sales
FROM Patient_Orders
WHERE Order_Date >= DATE '2026-01-01'
AND Order_Date < DATE '2027-01-01';
```

Explanation:

This query calculates the total recorded order value for the year 2026 using date conditions.

Output:

```
SQL> SELECT NVL(SUM(Total_Amount), 0) AS Period_Sales
  2  FROM Patient_Orders
  3  WHERE Order_Date >= DATE '2026-01-01'
  4  AND Order_Date < DATE '2027-01-01';

PERIOD_SALES
------------
        45.5
```

6. PATIENT PURCHASE ANALYSIS

6.1 Total Number of Orders Placed by Each Patient

SQL Query:

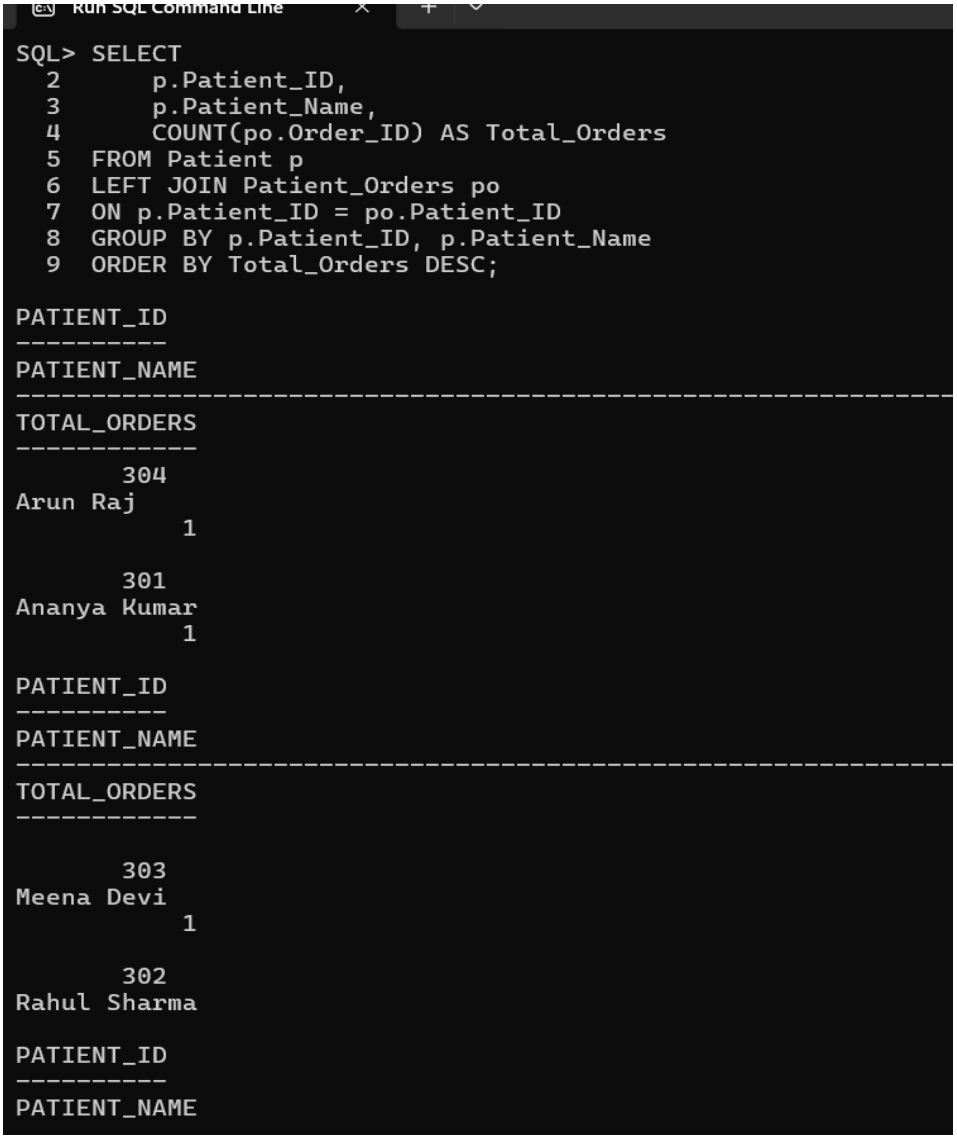
```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    COUNT(po.Order_ID) AS Total_Orders
FROM Patient p
LEFT JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
```

```
GROUP BY p.Patient_ID, p.Patient_Name
ORDER BY Total_Orders DESC;
```

Explanation:

This query calculates the number of orders placed by each patient. LEFT JOIN includes patients who have not placed any orders.

Output:



```
------------------------------
TOTAL_ORDERS
------------
           1

       305
Kavitha S
           0
```

6.2 Total Amount Spent by Each Patient

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    NVL(SUM(po.Total_Amount), 0)
        AS Total_Amount_Spent
FROM Patient p
LEFT JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
GROUP BY p.Patient_ID, p.Patient_Name
ORDER BY Total_Amount_Spent DESC;
```

Explanation:

This query calculates each patient's total order amount. NVL() displays zero for patients without orders.

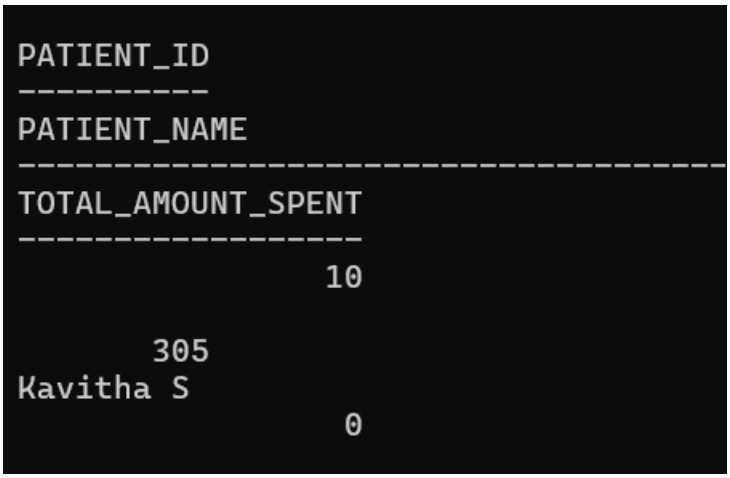
Output:

```
SQL> SELECT
  2      p.Patient_ID,
  3      p.Patient_Name,
  4      NVL(SUM(po.Total_Amount), 0) AS Total_Amount_Spent
  5  FROM Patient p
  6  LEFT JOIN Patient_Orders po
  7  ON p.Patient_ID = po.Patient_ID
  8  GROUP BY p.Patient_ID, p.Patient_Name
  9  ORDER BY Total_Amount_Spent DESC;

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------
TOTAL_AMOUNT_SPENT
------------------
       301
Ananya Kumar
                13

       304
Arun Raj
                12

PATIENT_ID
```



6.3 Average Spending per Patient

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    NVL(ROUND(AVG(po.Total_Amount), 2), 0)
        AS Average_Spending
FROM Patient p
LEFT JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
GROUP BY p.Patient_ID, p.Patient_Name
ORDER BY Average_Spending DESC;
```

Explanation:

This query calculates the average order amount for each patient using the AVG() function.

Output:

```
SQL> SELECT
  2      p.Patient_ID,
  3      p.Patient_Name,
  4      NVL(ROUND(AVG(po.Total_Amount), 2), 0)
  5          AS Average_Spending
  6  FROM Patient p
  7  LEFT JOIN Patient_Orders po
  8  ON p.Patient_ID = po.Patient_ID
  9  GROUP BY p.Patient_ID, p.Patient_Name
 10  ORDER BY Average_Spending DESC;

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
AVERAGE_SPENDING
----------------
       301
Ananya Kumar
              13

       304
Arun Raj
              12

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
AVERAGE_SPENDING
----------------

       302
Rahul Sharma
            10.5

       303
Meena Devi

PATIENT_ID
```

```
PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------
AVERAGE_SPENDING
----------------
              10

       305
Kavitha S
               0
```

6.4 Patients With Maximum Purchase Amount

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    SUM(po.Total_Amount) AS Total_Spending
FROM Patient p
JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
GROUP BY p.Patient_ID, p.Patient_Name
HAVING SUM(po.Total_Amount) = (
    SELECT MAX(SUM(Total_Amount))
    FROM Patient_Orders
    GROUP BY Patient_ID
);
```

Explanation:

This query identifies the patient or patients having the maximum total order amount using aggregate functions and a subquery.

Output:

```
SQL> SELECT
  2      p.Patient_ID,
  3      p.Patient_Name,
  4      SUM(po.Total_Amount) AS Total_Spending
  5  FROM Patient p
  6  JOIN Patient_Orders po
  7  ON p.Patient_ID = po.Patient_ID
  8  GROUP BY p.Patient_ID, p.Patient_Name
  9  HAVING SUM(po.Total_Amount) = (
 10      SELECT MAX(SUM(Total_Amount))
 11      FROM Patient_Orders
 12      GROUP BY Patient_ID
 13  );

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------
TOTAL_SPENDING
--------------
       301
Ananya Kumar
            13
```

6.5 Patients With Fewer Purchases

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    COUNT(po.Order_ID) AS Number_Of_Orders
FROM Patient p
LEFT JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
GROUP BY p.Patient_ID, p.Patient_Name
HAVING COUNT(po.Order_ID) <= 1
ORDER BY Number_Of_Orders;
```

Explanation:

This query identifies patients who have placed zero or one order.

Output:

```
SQL> SELECT
  2      p.Patient_ID,
  3      p.Patient_Name,
  4      COUNT(po.Order_ID) AS Number_Of_Orders
  5  FROM Patient p
  6  LEFT JOIN Patient_Orders po
  7  ON p.Patient_ID = po.Patient_ID
  8  GROUP BY p.Patient_ID, p.Patient_Name
  9  HAVING COUNT(po.Order_ID) <= 1
 10  ORDER BY Number_Of_Orders;

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------
NUMBER_OF_ORDERS
---------------
       305
Kavitha S
               0

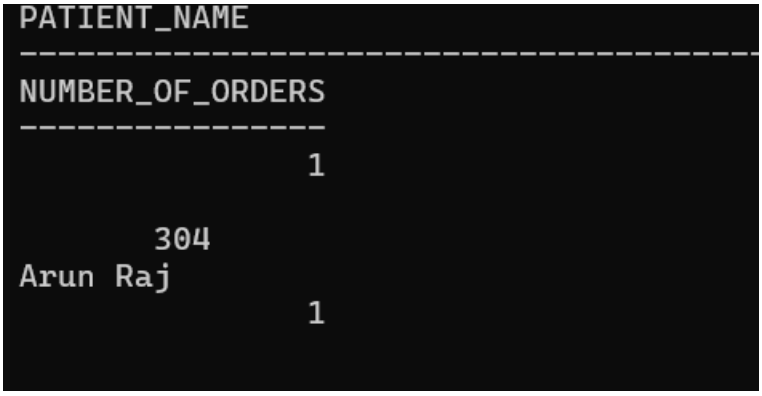
       301
Ananya Kumar
               1

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------
NUMBER_OF_ORDERS
---------------

       303
Meena Devi
               1

       302
Rahul Sharma

PATIENT_ID
----------
```



7. TOP PATIENTS BASED ON PURCHASE AMOUNT

7.1 Top Five Patients Based on Total Spending

SQL Query:

```
SELECT *
FROM (
    SELECT
        p.Patient_ID,
        p.Patient_Name,
        SUM(po.Total_Amount) AS Total_Spending
    FROM Patient p
    JOIN Patient_Orders po
    ON p.Patient_ID = po.Patient_ID
    GROUP BY p.Patient_ID, p.Patient_Name
    ORDER BY Total_Spending DESC
)
WHERE ROWNUM <= 5;
```

Explanation:

This query identifies the top five patients based on total order value. ROWNUM limits the result to five records.

Output:

```
SQL> SELECT *
  2  FROM (
  3      SELECT
  4          p.Patient_ID,
  5          p.Patient_Name,
  6          SUM(po.Total_Amount) AS Total_Spending
  7      FROM Patient p
  8      JOIN Patient_Orders po
  9      ON p.Patient_ID = po.Patient_ID
 10      GROUP BY p.Patient_ID, p.Patient_Name
 11      ORDER BY Total_Spending DESC
 12  )
 13  WHERE ROWNUM <= 5;

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
TOTAL_SPENDING
--------------
       301
Ananya Kumar
           13

       304
Arun Raj
           12

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
TOTAL_SPENDING
--------------

       302
Rahul Sharma
          10.5

       303
```

```
       303
Meena Devi

PATIENT_ID
----------
PATIENT_NAME
--------------------------------
TOTAL_SPENDING
--------------
            10
```

7.2 Patients With Maximum Number of Orders

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    COUNT(po.Order_ID) AS Number_Of_Orders
FROM Patient p
JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
GROUP BY p.Patient_ID, p.Patient_Name
HAVING COUNT(po.Order_ID) = (
    SELECT MAX(Order_Count)
    FROM (
        SELECT COUNT(*) AS Order_Count
        FROM Patient_Orders
        GROUP BY Patient_ID
    )
);
```

Explanation:

This query identifies patients who have placed the highest number of orders.

Output:

```
SQL> SELECT
  2      p.Patient_ID,
  3      p.Patient_Name,
  4      COUNT(po.Order_ID) AS Number_Of_Orders
  5  FROM Patient p
  6  JOIN Patient_Orders po
  7  ON p.Patient_ID = po.Patient_ID
  8  GROUP BY p.Patient_ID, p.Patient_Name
  9  HAVING COUNT(po.Order_ID) = (
 10      SELECT MAX(Order_Count)
 11      FROM (
 12          SELECT COUNT(*) AS Order_Count
 13          FROM Patient_Orders
 14          GROUP BY Patient_ID
 15      )
 16  );

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS
----------------
       304
Arun Raj
               1

       301
Ananya Kumar
               1

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS
----------------

       302
Rahul Sharma
```

```
       303
Meena Devi

PATIENT_ID
----------
PATIENT_NAME
----------------------
NUMBER_OF_ORDERS
---------------
              1
```

7.3 Frequent Patients

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    COUNT(po.Order_ID) AS Number_Of_Orders
FROM Patient p
JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
GROUP BY p.Patient_ID, p.Patient_Name
HAVING COUNT(po.Order_ID) >= 2
ORDER BY Number_Of_Orders DESC;
```

Explanation:

This query identifies patients who have placed two or more orders.

Output:

```
SQL> SELECT
  2      p.Patient_ID,
  3      p.Patient_Name,
  4      COUNT(po.Order_ID) AS Number_Of_Orders
  5  FROM Patient p
  6  JOIN Patient_Orders po
  7  ON p.Patient_ID = po.Patient_ID
  8  GROUP BY p.Patient_ID, p.Patient_Name
  9  HAVING COUNT(po.Order_ID) >= 2
 10  ORDER BY Number_Of_Orders DESC;

no rows selected
```

7.4 High-Value Patients

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    COUNT(po.Order_ID) AS Number_Of_Orders,
    SUM(po.Total_Amount) AS Total_Spending
FROM Patient p
JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
GROUP BY p.Patient_ID, p.Patient_Name
HAVING SUM(po.Total_Amount) > 20
ORDER BY Total_Spending DESC;
```

Explanation:

This query identifies patients whose total recorded order value exceeds ₹20. This threshold is used for demonstration purposes.

Output:

```
SQL> SELECT
  2      m.Medicine_ID,
  3      m.Medicine_Name,
  4      SUM(od.Quantity) AS Total_Quantity_Sold
  5  FROM Medicine m
  6  JOIN Order_Details od
  7  ON m.Medicine_ID = od.Medicine_ID
  8  GROUP BY m.Medicine_ID, m.Medicine_Name
  9  ORDER BY Total_Quantity_Sold DESC;

MEDICINE_ID
-----------
MEDICINE_NAME
--------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
        203
Cetirizine
                  5

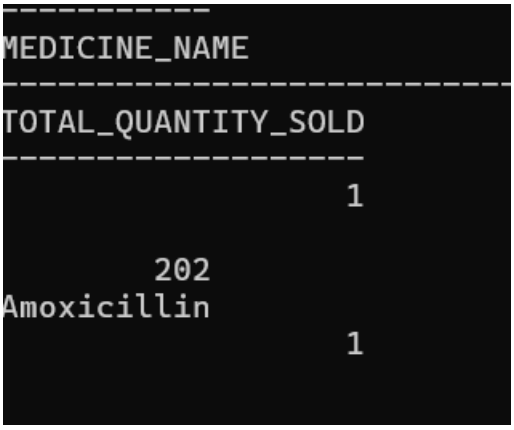
        201
Paracetamol
                  2

MEDICINE_ID
-----------
MEDICINE_NAME
--------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

        204
Omeprazole
                  2

        205
Azithromycin

MEDICINE_ID
-----------
```



8. IDENTIFY BEST-SELLING MEDICINES

8.1 Medicines With Maximum Sales Quantity

SQL Query:

```
SELECT
    m.Medicine_ID,
    m.Medicine_Name,
    SUM(od.Quantity) AS Total_Quantity_Sold
FROM Medicine m
JOIN Order_Details od
ON m.Medicine_ID = od.Medicine_ID
GROUP BY m.Medicine_ID, m.Medicine_Name
ORDER BY Total_Quantity_Sold DESC;
```

Explanation:

This query calculates the total quantity ordered for each medicine and arranges the results in descending order.

Output:

```
SQL> SELECT
  2      m.Medicine_ID,
  3      m.Medicine_Name,
  4      SUM(od.Quantity) AS Quantity_Sold,
  5      SUM(od.Quantity * od.Price) AS Total_Revenue
  6  FROM Medicine m
  7  JOIN Order_Details od
  8  ON m.Medicine_ID = od.Medicine_ID
  9  GROUP BY m.Medicine_ID, m.Medicine_Name
 10  ORDER BY Total_Revenue DESC;

MEDICINE_ID
-----------
MEDICINE_NAME
--------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
        203
Cetirizine
            5          17.5

        205
Azithromycin
            1            12

MEDICINE_ID
-----------
MEDICINE_NAME
--------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------

        204
Omeprazole
            2            10

        202
Amoxicillin

MEDICINE_ID
```

```
MEDICINE_ID
-----------
MEDICINE_NAME
----------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
            1             8

        201
Paracetamol
            2             5
```

8.2 Medicines Generating Highest Revenue

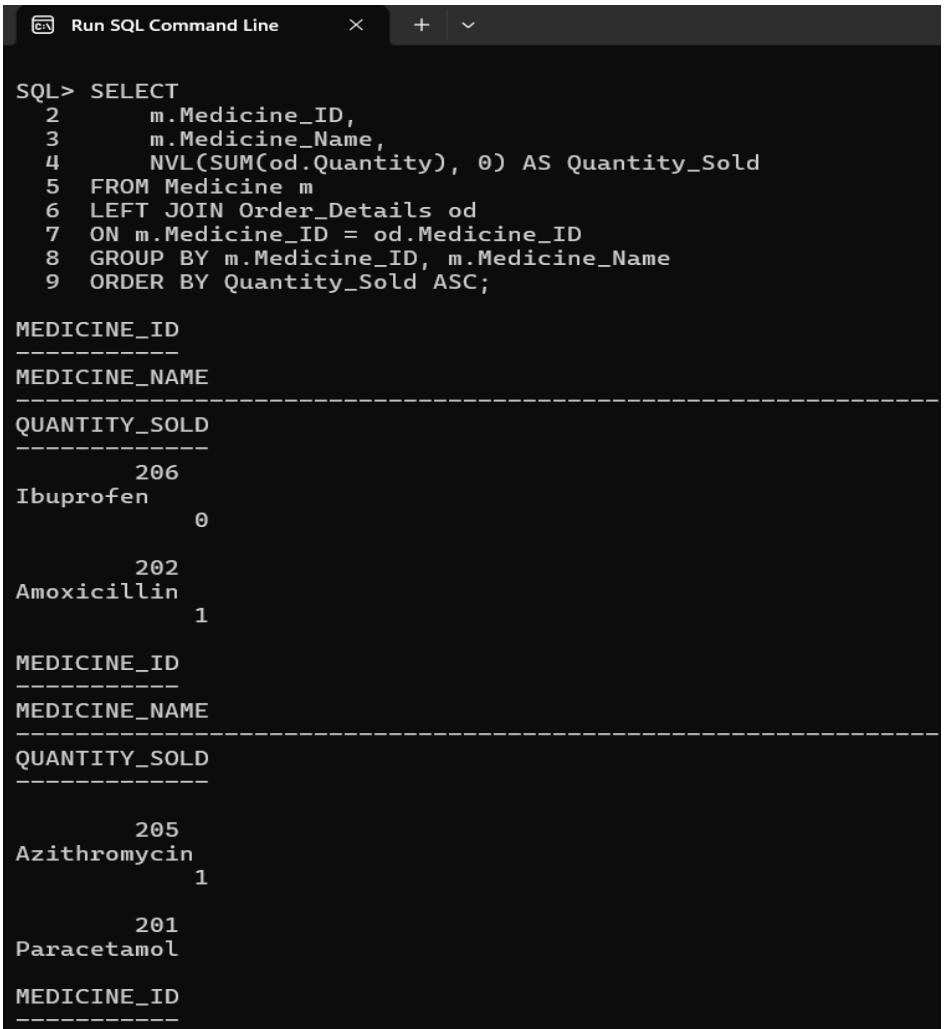
SQL Query:

```
SELECT
    m.Medicine_ID,
    m.Medicine_Name,
    SUM(od.Quantity) AS Quantity_Sold,
    SUM(od.Quantity * od.Price) AS Total_Revenue
FROM Medicine m
JOIN Order_Details od
ON m.Medicine_ID = od.Medicine_ID
GROUP BY m.Medicine_ID, m.Medicine_Name
ORDER BY Total_Revenue DESC;
```

Explanation:

This query calculates the total recorded sales value of each medicine by multiplying quantity and price.

Output:



```
MEDICINE_ID
-----------
MEDICINE_NAME
----------------------------------------
QUANTITY_SOLD
-------------
            2

        204
Omeprazole
            2

        203

MEDICINE_ID
-----------
MEDICINE_NAME
----------------------------------------
QUANTITY_SOLD
-------------
Cetirizine
            5


6 rows selected.
```

8.3 Least-Selling Medicines

SQL Query:

```
SELECT
    m.Medicine_ID,
    m.Medicine_Name,
    NVL(SUM(od.Quantity), 0) AS Quantity_Sold
FROM Medicine m
LEFT JOIN Order_Details od
ON m.Medicine_ID = od.Medicine_ID
GROUP BY m.Medicine_ID, m.Medicine_Name
ORDER BY Quantity_Sold ASC;
```

Explanation:

This query identifies medicines with the lowest ordered quantities. LEFT JOIN also includes medicines with no orders.

Output:

```
SQL> SELECT
  2      m.Medicine_ID,
  3      m.Medicine_Name,
  4      m.Unit_Price,
  5      NVL(SUM(od.Quantity), 0) AS Quantity_Sold
  6  FROM Medicine m
  7  LEFT JOIN Order_Details od
  8  ON m.Medicine_ID = od.Medicine_ID
  9  GROUP BY m.Medicine_ID, m.Medicine_Name, m.Unit_Price
 10  HAVING NVL(SUM(od.Quantity), 0) <= 1
 11  ORDER BY Quantity_Sold ASC;

MEDICINE_ID
-----------
MEDICINE_NAME
--------------------------------------------------------------
UNIT_PRICE QUANTITY_SOLD
---------- -------------
        206
Ibuprofen
         4             0

        202
Amoxicillin
         8             1

MEDICINE_ID
-----------
MEDICINE_NAME
--------------------------------------------------------------
UNIT_PRICE QUANTITY_SOLD
---------- -------------

        205
Azithromycin
        12             1
```

8.4 Medicines Requiring Sales Promotion

SQL Query:

```
SELECT
    m.Medicine_ID,
    m.Medicine_Name,
    m.Unit_Price,
    NVL(SUM(od.Quantity), 0) AS Quantity_Sold
FROM Medicine m
LEFT JOIN Order_Details od
ON m.Medicine_ID = od.Medicine_ID
GROUP BY
    m.Medicine_ID,
    m.Medicine_Name,
    m.Unit_Price
HAVING NVL(SUM(od.Quantity), 0) <= 1
ORDER BY Quantity_Sold ASC;
```

Explanation:

This query identifies medicines with one or fewer units ordered. It demonstrates low-sales analysis for database reporting and does not recommend increasing medicine consumption.

Output:

```
SQL> SELECT
  2      m.Category,
  3      SUM(od.Quantity * od.Price) AS Total_Sales
  4  FROM Medicine m
  5  JOIN Order_Details od
  6  ON m.Medicine_ID = od.Medicine_ID
  7  GROUP BY m.Category
  8  ORDER BY Total_Sales DESC;

CATEGORY                                           TOTAL_SALES
-------------------------------------------------- -----------
Antibiotic                                                  20
Allergy                                                   17.5
Gastric                                                     10
Pain Relief                                                  5
```

9. CATEGORY-WISE MEDICINE SALES ANALYSIS

9.1 Total Sales Generated by Each Medicine Category

SQL Query:

```
SELECT
    m.Category,
    SUM(od.Quantity * od.Price) AS Total_Sales
FROM Medicine m
JOIN Order_Details od
ON m.Medicine_ID = od.Medicine_ID
GROUP BY m.Category
ORDER BY Total_Sales DESC;
```

Explanation:

This query calculates the total recorded sales value for each medicine category.

Output:

```
SQL> SELECT
  2      m.Category,
  3      SUM(od.Quantity) AS Medicines_Sold
  4  FROM Medicine m
  5  JOIN Order_Details od
  6  ON m.Medicine_ID = od.Medicine_ID
  7  GROUP BY m.Category
  8  ORDER BY Medicines_Sold DESC;

CATEGORY                                           MEDICINES_SOLD
-------------------------------------------------- --------------
Allergy                                                         5
Gastric                                                         2
Pain Relief                                                     2
Antibiotic                                                      2
```

9.2 Number of Medicine Units Sold in Each Category

SQL Query:

```
SELECT
    m.Category,
    SUM(od.Quantity) AS Medicines_Sold
FROM Medicine m
JOIN Order_Details od
ON m.Medicine_ID = od.Medicine_ID
GROUP BY m.Category
ORDER BY Medicines_Sold DESC;
```

Explanation:

This query calculates the total number of medicine units ordered in each category.

Output:

```
SQL> SELECT
  2      m.Category,
  3      SUM(od.Quantity * od.Price) AS Total_Revenue
  4  FROM Medicine m
  5  JOIN Order_Details od
  6  ON m.Medicine_ID = od.Medicine_ID
  7  GROUP BY m.Category
  8  HAVING SUM(od.Quantity * od.Price) = (
  9      SELECT MAX(Category_Revenue)
 10      FROM (
 11          SELECT SUM(od2.Quantity * od2.Price)
 12              AS Category_Revenue
 13          FROM Medicine m2
 14          JOIN Order_Details od2
 15          ON m2.Medicine_ID = od2.Medicine_ID
 16          GROUP BY m2.Category
 17      )
 18  );

CATEGORY                                           TOTAL_REVENUE
-------------------------------------------------- -------------
Antibiotic                                                    20
```

9.3 Highest Revenue-Generating Medicine Category

SQL Query:

```
SELECT
    m.Category,
    SUM(od.Quantity * od.Price) AS Total_Revenue
FROM Medicine m
JOIN Order_Details od
ON m.Medicine_ID = od.Medicine_ID
GROUP BY m.Category
HAVING SUM(od.Quantity * od.Price) = (
    SELECT MAX(Category_Revenue)
    FROM (
        SELECT SUM(od2.Quantity * od2.Price)
            AS Category_Revenue
        FROM Medicine m2
        JOIN Order_Details od2
        ON m2.Medicine_ID = od2.Medicine_ID
        GROUP BY m2.Category
    )
);
```

Explanation:

This query identifies the medicine category generating the highest recorded sales value using GROUP BY, HAVING, and a subquery.

Output:

```
SQL> SELECT
  2      m.Category,
  3      ROUND(AVG(od.Quantity * od.Price), 2)
  4          AS Average_Sales
  5  FROM Medicine m
  6  JOIN Order_Details od
  7  ON m.Medicine_ID = od.Medicine_ID
  8  GROUP BY m.Category
  9  ORDER BY Average_Sales DESC;

CATEGORY                                           AVERAGE_SALES
-------------------------------------------------- -------------
Allergy                                                     17.5
Gastric                                                       10
Antibiotic                                                    10
Pain Relief                                                    5
```

9.4 Average Sales per Medicine Category

SQL Query:

```
SELECT
    m.Category,
    ROUND(AVG(od.Quantity * od.Price), 2)
        AS Average_Sales
FROM Medicine m
JOIN Order_Details od
ON m.Medicine_ID = od.Medicine_ID
GROUP BY m.Category
ORDER BY Average_Sales DESC;
```

Explanation:

This query calculates the average order-line value for each medicine category.

Output:

```
SQL> SELECT
  2      NVL(SUM(Total_Amount), 0) AS Total_Sales,
  3      COUNT(Order_ID) AS Number_Of_Orders,
  4      ROUND(AVG(Total_Amount), 2) AS Average_Order_Value,
  5      MAX(Total_Amount) AS Highest_Order_Value
  6  FROM Patient_Orders;

TOTAL_SALES NUMBER_OF_ORDERS AVERAGE_ORDER_VALUE HIGHEST_ORDER_VALUE
----------- ---------------- ------------------- -------------------
       45.5                4               11.38                  13

SQL>
```

10. GENERATE HOSPITAL ANALYTICS REPORTS

10.1 Report 1 – Hospital Sales Performance Report

SQL Query:

```
SELECT
    NVL(SUM(Total_Amount), 0) AS Total_Sales,
    COUNT(Order_ID) AS Number_Of_Orders,
    ROUND(AVG(Total_Amount), 2)
        AS Average_Order_Value,
    MAX(Total_Amount) AS Highest_Order_Value
FROM Patient_Orders;
```

Explanation:

This report displays total recorded sales value, total number of orders, average order value, and highest order value.

Output:

```
      45.5                4               11.38                   13

SQL> SELECT
  2      p.Patient_Name,
  3      COUNT(po.Order_ID) AS Number_Of_Orders,
  4      NVL(SUM(po.Total_Amount), 0) AS Total_Spending,
  5      COUNT(po.Order_ID) AS Purchase_Frequency
  6  FROM Patient p
  7  LEFT JOIN Patient_Orders po
  8  ON p.Patient_ID = po.Patient_ID
  9  GROUP BY p.Patient_ID, p.Patient_Name
 10  ORDER BY Total_Spending DESC;

PATIENT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_SPENDING PURCHASE_FREQUENCY
---------------- -------------- ------------------
Ananya Kumar
               1             13                  1

Arun Raj
               1             12                  1

Rahul Sharma
               1           10.5                  1


PATIENT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_SPENDING PURCHASE_FREQUENCY
---------------- -------------- ------------------
Meena Devi
               1             10                  1
```

10.2 Report 2 – Patient Analytics Report

SQL Query:

```
SELECT
    p.Patient_Name,
    COUNT(po.Order_ID) AS Number_Of_Orders,
    NVL(SUM(po.Total_Amount), 0) AS Total_Spending,
    COUNT(po.Order_ID) AS Purchase_Frequency
FROM Patient p
LEFT JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
GROUP BY p.Patient_ID, p.Patient_Name
ORDER BY Total_Spending DESC;
```

Explanation:

This report displays patient names, number of orders, total recorded order value, and purchase frequency. Here, purchase frequency represents the number of orders recorded for each patient.

Output:

```
SQL> SELECT
  2       m.Medicine_Name,
  3       SUM(od.Quantity) AS Quantity_Sold,
  4       SUM(od.Quantity * od.Price) AS Revenue_Generated
  5  FROM Medicine m
  6  JOIN Order_Details od
  7  ON m.Medicine_ID = od.Medicine_ID
  8  GROUP BY m.Medicine_ID, m.Medicine_Name
  9  ORDER BY Revenue_Generated DESC;

MEDICINE_NAME
--------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- ------------------
Cetirizine
            5               17.5

Azithromycin
            1                 12

Omeprazole
            2                 10


MEDICINE_NAME
--------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- ------------------
Amoxicillin
            1                  8
```

10.3 Report 3 – Medicine Performance Report

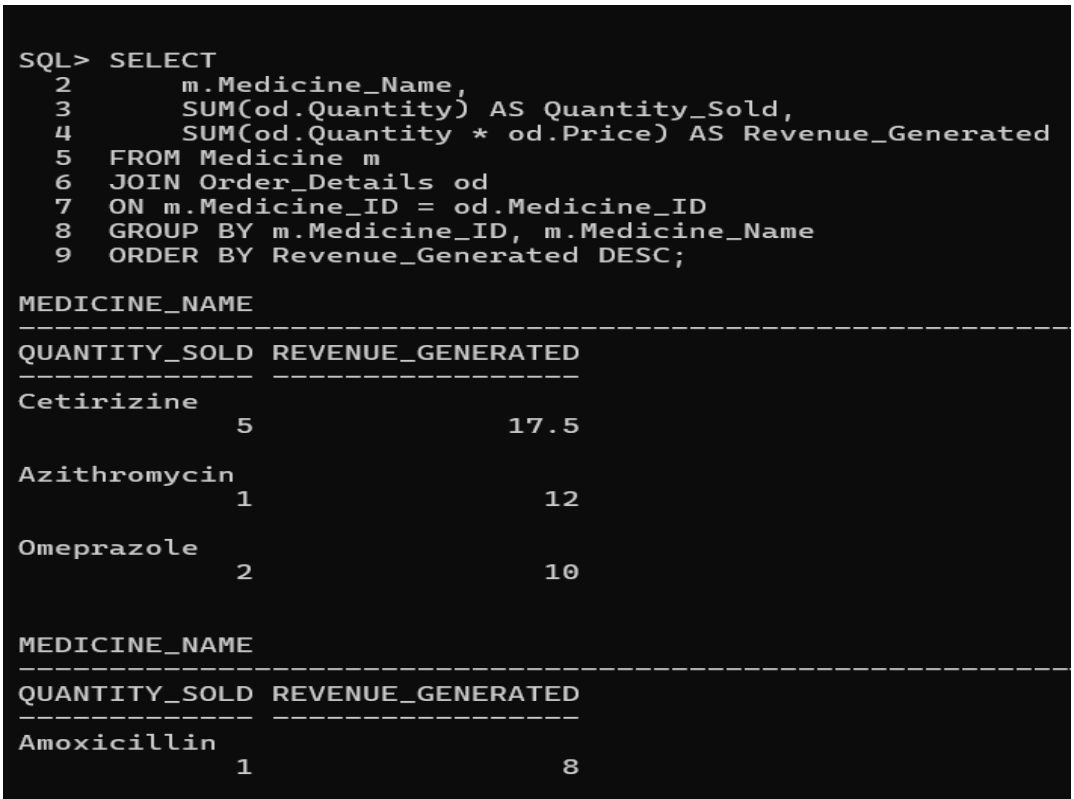
SQL Query:

```
SELECT
    m.Medicine_Name,
    SUM(od.Quantity) AS Quantity_Sold,
    SUM(od.Quantity * od.Price)
        AS Revenue_Generated
FROM Medicine m
JOIN Order_Details od
ON m.Medicine_ID = od.Medicine_ID
GROUP BY m.Medicine_ID, m.Medicine_Name
ORDER BY Revenue_Generated DESC;
```

Explanation:

This report displays medicine names, quantities ordered, and the corresponding recorded sales values.

Output:



```
SQL> SELECT
  2      m.Medicine_Name,
  3      SUM(od.Quantity) AS Quantity_Sold,
  4      SUM(od.Quantity * od.Price) AS Revenue_Generated
  5  FROM Medicine m
  6  JOIN Order_Details od
  7  ON m.Medicine_ID = od.Medicine_ID
  8  GROUP BY m.Medicine_ID, m.Medicine_Name
  9  ORDER BY Revenue_Generated DESC;

MEDICINE_NAME
--------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
Cetirizine
            5              17.5

Azithromycin
            1                12

Omeprazole
            2                10


MEDICINE_NAME
--------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
Amoxicillin
            1                 8
```

10.4 Report 4 – Medicine Category Analysis Report

SQL Query:

```
SELECT
    m.Category,
    SUM(od.Quantity) AS Total_Medicines_Sold,
    SUM(od.Quantity * od.Price) AS Total_Revenue
FROM Medicine m
JOIN Order_Details od
ON m.Medicine_ID = od.Medicine_ID
GROUP BY m.Category
ORDER BY Total_Revenue DESC;
```

Explanation:

This report displays each medicine category, the total quantity ordered, and the total recorded sales value.

Output:

```
SQL> SELECT
  2      m.Category,
  3      SUM(od.Quantity) AS Total_Medicines_Sold,
  4      SUM(od.Quantity * od.Price) AS Total_Revenue
  5  FROM Medicine m
  6  JOIN Order_Details od
  7  ON m.Medicine_ID = od.Medicine_ID
  8  GROUP BY m.Category
  9  ORDER BY Total_Revenue DESC;

CATEGORY                                           TOTAL_MEDICINES_SOLD
-------------------------------------------------- --------------------
TOTAL_REVENUE
-------------
Antibiotic                                                            2
           20

Allergy                                                               5
         17.5

Gastric                                                               2
           10


CATEGORY                                           TOTAL_MEDICINES_SOLD
-------------------------------------------------- --------------------
TOTAL_REVENUE
-------------
```

11. RESULT

The Hospital Sales and Patient Analytics System module demonstrates the use of SQL aggregate functions and analytical queries in the CityCare Hospital Management System.

The module includes total sales reports, patient purchase analysis, identification of top purchasing patients, medicine sales analysis, category-wise sales analysis, and hospital analytics reports.

The results of the executed SQL queries are presented in the corresponding SQL*Plus screenshots.

12. CONCLUSION

The Hospital Sales and Patient Analytics System provides a structured approach to analysing patient orders, medicine purchases, and recorded sales values.

The implementation demonstrates the practical use of COUNT(), SUM(), AVG(), MIN(), MAX(), NVL(), JOIN, GROUP BY, HAVING, ORDER BY, and subqueries in Oracle SQL.

Through these operations, the system can generate useful reports about patient purchasing behaviour, medicine performance, and category-wise sales.

Thus, Week 9 demonstrates how database analytics can support reporting and decision-making in the CityCare Hospital Management System.